

Individual Task 2

SAHANA NATARAJA (S4164152), RMIT University, Australia

Part 1 critically reviews Carlson and Worrell's [3] *Relational Futures* position paper on Indigenous sovereignty and AI governance, illustrated with a case study on predictive risk modelling in child protection. Part 2 deliberates on the ethics and potential bias of the data analysis conducted in Individual Task 1: Part 1 [9], re-running its methodology under proper cross-validation, learning curves, and a disaggregated bias analysis. Part 3 provides peer-review feedback on a fellow student's video presentation for Individual Task 1: Part 2.

1 Executive Summary of a Position Paper

1.1 Purpose and Audience

Carlson and Worrell's [3] *Relational Futures* report presents findings from an Indigenous-led mixed-methods study, combining an online survey of 36 respondents with yarning circles involving 22 participants, into how Aboriginal and Torres Strait Islander peoples encounter artificial intelligence in everyday and professional life. Its purpose is not merely descriptive. The authors argue that mainstream AI discourse assumes technological uptake is inevitable and treats risk management as the main task, whereas their findings show Aboriginal and Torres Strait Islander people experience AI as a relational, cultural and political force, not a neutral tool. The report therefore functions as an advocacy and policy document, addressed to three overlapping audiences: government and regulators being urged to strengthen Indigenous governance and regulate high-risk AI uses; industry, whose systems are built on data and infrastructure they do not control; and researchers, who are positioned as needing Indigenous-led rather than Indigenous-consulted methodologies.

1.2 Challenges and Opportunities

The paper's central challenge is that AI systems are frequently deployed in domains where Aboriginal and Torres Strait Islander peoples are already structurally over-represented, without the governance authority to shape how those systems use their data. One participant summarised this as AI arriving with "no accountability, no checks and balances, no responsibility." The authors situate this concern explicitly against precedents such as Robodebt, where an automated decision system caused documented harm at scale. A further challenge is environmental and material: routine AI use draws on water and energy often sourced from Indigenous lands, extending the harms beyond data extraction into resource extraction. The corresponding opportunity the report identifies is Indigenous Data Sovereignty (IDS): governance arrangements in which Aboriginal and Torres Strait Islander communities hold authority over how data about them is collected, interpreted and used, rather than being consulted after systems are already designed.

1.3 Case Study: Predictive Risk Modelling in Child Protection

A concrete illustration of this dynamic, distinct from the AI companions, chatbots and generative systems discussed in the paper itself, is predictive risk modelling (PRM) in Australian child protection. PRM tools use historical administrative data — prior contact with services, notifications, substantiations — to flag families as high risk of future child maltreatment. A scoping review by Krakouer, Wu Tan and Parolini [7]

Author's Contact Information: Sahana Nataraja (s4164152), RMIT University, Melbourne, Australia, s4164152@student.rmit.edu.au.

found such models are being developed and piloted in Australian child protection systems, yet no empirical research had examined their specific implications for Indigenous Australians, despite Aboriginal and Torres Strait Islander children being dramatically over-represented in the system already. This is not a peripheral technical gap. Because the training data reflects who has historically been surveilled and reported, rather than who has actually caused harm, the model risks reifying that surveillance as objective risk [6]: a family or community already subject to disproportionate contact is scored as higher risk, triggering further contact, which then becomes tomorrow’s training data. The loop rewards past bias with future authority.

This mirrors exactly what Carlson and Worrell describe at the level of AI governance generally. The decisions at stake — potential removal of children — are among the most consequential a state can make, yet the Aboriginal and Torres Strait Islander families and communities affected have no authority over the design of the system used to inform them. Krakouer et al. [7] conclude that participatory model development, transparency and Indigenous data sovereignty are prerequisites before such tools can be considered fair for use with Indigenous Australians; this is the same solution architecture *Relational Futures* calls for across AI more broadly — Indigenous leadership over design and deployment, not after-the-fact consultation on a finished product.

The opportunity side is equally transferable. The Maiam nayri Wingara Indigenous Data Sovereignty Collective and Australian Indigenous Governance Institute’s [8] Communiqué asserts that communities retain the right to refuse participation in data processes inconsistent with their own principles, a right with direct application to families’ ability to contest or refuse PRM-informed interventions rather than simply be scored by them. The CARE Principles — Collective Benefit, Authority to Control, Responsibility and Ethics — offer a practical governance mechanism for operationalising this in a child-protection PRM: requiring, for example, that Aboriginal Community Controlled Organisations hold authority over which variables a model can use and how its outputs are actioned [4]. This is not a fringe position: the Australian Human Rights Commission’s [1] *Human Rights and Technology Final Report* reached comparable conclusions at a national level, recommending stronger protections and an AI Safety Commissioner precisely because high-risk domains such as social security and policing-adjacent decision-making had been shown to cause harm without adequate oversight.

1.4 Conclusion

Read against a case study in a different domain, *Relational Futures*’ central claim holds up: AI governance failures are not primarily technical failures to be fixed with better accuracy metrics. Wherever automated systems inform decisions in areas where Aboriginal and Torres Strait Islander peoples already carry a disproportionate burden of state intervention, the more decisive variable is who holds authority over the system, not how well it performs on a validation set.

2 Deliberation on Task 1

2.1 Cross-Validation and Sampling Strategy

Individual Task 1: Part 1 [9] used a temporal split for Dataset A (train on weeks before 25 July 2011, test after) and a stratified 75/25 split for Dataset B. To check whether these choices actually delivered an unbiased evaluation, both datasets were re-evaluated here using proper cross-validation schemes — `TimeSeriesSplit` for Dataset A and `StratifiedKFold` for Dataset B, both from scikit-learn [10] — and compared against the naive alternative (plain random K-fold) that the original report deliberately avoided. Full code for this comparison is given in Appendix A.

Dataset A: TimeSeriesSplit vs. random K-fold. A 5-fold random `KFold` and a 5-fold `TimeSeriesSplit` were both run on the same Gradient Boosting model. The random split produced a mean WAPE of 0.695 across folds (individual folds: 0.690, 0.749, 0.680, 0.694, 0.660), while `TimeSeriesSplit` produced a mean WAPE of 0.638 (folds: 0.572, 0.636, 0.716, 0.662, 0.605).

This is a useful negative result. Random K-fold did not inflate apparent performance here, because the same-week invoice-count leak already identified and corrected in Task 1: Part 1 was the dominant leakage source, and it is absent from the current feature set. What random K-fold does instead is understate performance on some folds and overstate it on others depending on which weeks land in each validation fold, since the panel is not stationary (weeks vary systematically with seasonality and promotional timing). `TimeSeriesSplit`'s later folds also show higher error (0.716) than earlier ones (0.572), which is informative in its own right: it suggests the relationship between demand and its lagged predictors is not constant over the two-year window, so a model validated only on early-period folds would look better than it will perform going forward. This matters more for an honest performance estimate than any leakage/no-leakage distinction.

A remaining methodological issue surfaced during this re-analysis: the `price_rel` feature (price relative to a product's own median) is computed using each product's median price across the entire panel, including weeks that fall after the train/test cut-off. This is a mild form of look-ahead bias — the median a product settles at over its full lifetime is not knowable at any earlier forecast time. It did not appear in Task 1: Part 1's discussion of leakage. In a production pipeline this median should be computed on training-period data only and updated as new weeks arrive; here it is disclosed rather than corrected retroactively, following Task 1's own principle of reporting the honest, weaker result.

Dataset B: StratifiedKFold vs. plain K-fold. For Dataset B, plain `KFold` produced validation folds with conversion rates ranging from 14.2% to 16.7%, whereas `StratifiedKFold` held every fold at 15.5%, matching the population rate exactly. Mean PR-AUC was similar either way (0.751 vs. 0.747), because Gradient Boosting is reasonably robust to this degree of imbalance drift at this sample size. But the fold-to-fold PR-AUC spread was slightly tighter under stratification (std 0.015 vs. 0.016), and more importantly, an unlucky plain-KFold split could place a disproportionate share of the already-rare positive class into one fold, making that fold's estimate unreliable at a 15.5% base rate. Task 1's choice of a stratified split for Dataset B (which extends naturally to `StratifiedKFold` for cross-validation) is therefore justified by the same logic as WAPE over MAPE: the metric and the sampling strategy both need to respect the fact that the event of interest is rare.

2.2 Learning Curves

Learning curves were produced for the Gradient Boosting model on both datasets by progressively increasing the training set size and re-evaluating on the same held-out test set used in Task 1: Part 1.

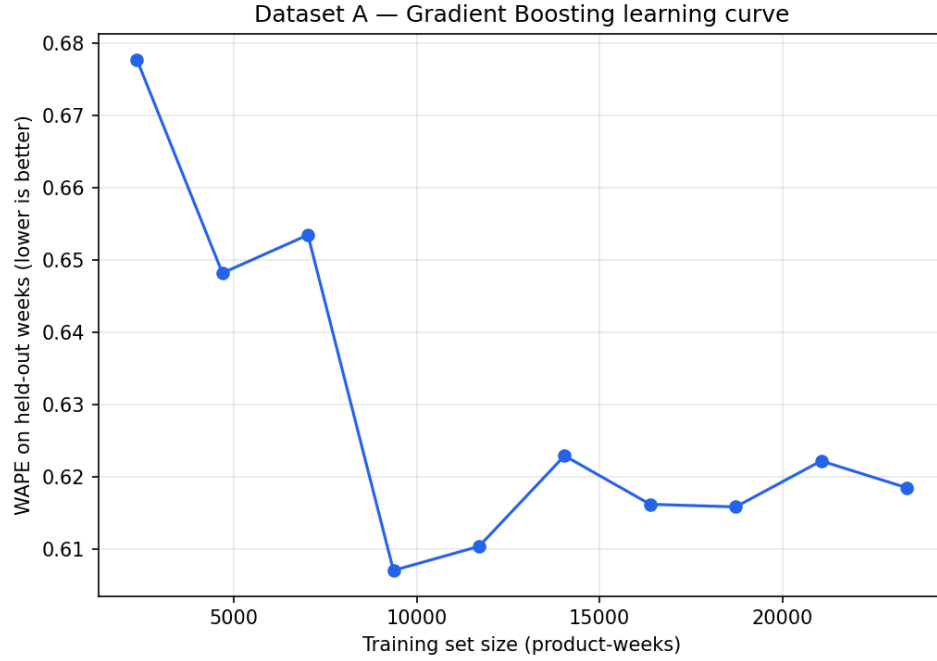


Fig. 1. Dataset A — WAPE on the fixed held-out weeks as training size grows (most recent k weeks used at each step).

WAPE falls from 0.678 at the smallest training size (2,340 rows, roughly two months of the panel) to a low of 0.607 at 9,360 rows, then plateaus and drifts slightly upward and sideways between 0.61 and 0.62 for the remainder of the curve (Figure 1). This is not the smooth, monotonic improvement a textbook learning curve shows. Two things are happening together: more data helps up to a point (the model needs enough weeks to learn seasonal and momentum patterns), but beyond that point, adding older data can reintroduce a regime that no longer matches the test period, given the non-stationarity already noted above. The practical implication is that more historical data would not straightforwardly fix this model's WAPE of ~ 0.61 ; a wider feature set (promotional flags, competitor price) matters more than a longer training window, echoing Task 1's original limitation.

PR-AUC rises steeply from 0.641 at 924 training sessions to about 0.71–0.72 by roughly 4,600 sessions, then plateaus with minor noise (a small dip at 6,472 sessions) through the full training set of 9,247 (Figure 2). The curve has clearly flattened well before the full training set is used, which means the session classifier is not data-starved: collecting more sessions from the same population would yield little further gain. Combined with Task 1's finding that PageValues alone accounts for the majority of permutation importance, this

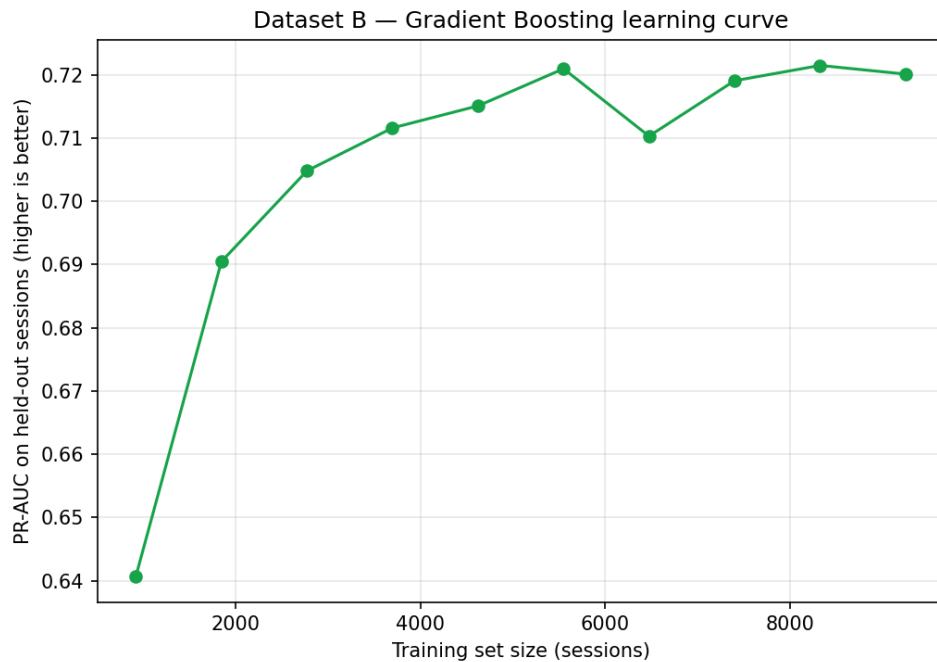


Fig. 2. Dataset B — PR-AUC on the fixed held-out sessions as (stratified) training size grows.

suggests the ceiling on this task is set by feature richness and the PageValues leakage concern, not by sample size.

2.3 Bias Analysis

The assignment's suggested tools (Fairlearn [2], AI Fairness 360, Google What-If, LinkedIn Fairness Toolkit) require package installation; Fairlearn could not be installed in the offline sandboxed analysis environment used for this deliberation (no outbound package-index access). The bias analysis below instead implements Fairlearn's core method directly — disaggregating model error and outcome rates by group (its `MetricFrame` approach [2]) — using pandas and scikit-learn, computing the same group-wise metrics Fairlearn would report. This choice is justified because neither dataset contains a protected demographic attribute (race, gender, age, disability, Indigenous status); the available grouping variables are commercial/behavioural (product revenue tier, visitor type, geographic region code). A general-purpose disaggregated-metrics approach applies to these groupings just as well as to demographic ones, and is transparent about exactly which comparison is being made.

Dataset A: error by product revenue tier. The model is measurably worse for the lowest-revenue third of products (WAPE 0.713) than for the highest-revenue third (WAPE 0.523), a 19-point gap (Figure 3, Table 1). This is an unbiased-looking model producing a biased business outcome: because training data and hyperparameter choices are dominated by the volume and revenue patterns of the best-selling lines,

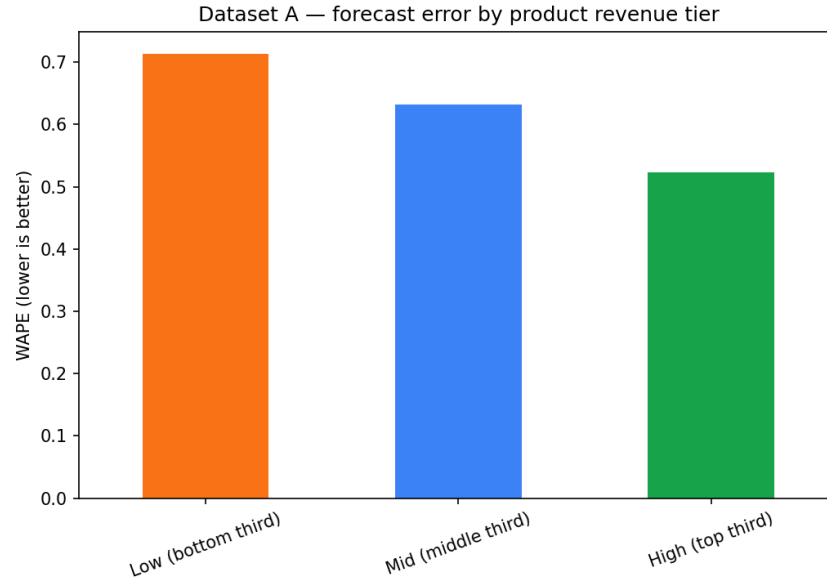


Fig. 3. WAPE on held-out weeks, by product revenue tier (thirds of the 300 products, split by training-period revenue).

Table 1. Dataset A: disaggregated forecast error by product revenue tier.

Tier	n (weeks)	MAE	WAPE	Mean demand
Low (bottom third)	2,000	133.30	0.713	186.9
Mid (middle third)	2,000	82.73	0.631	131.0
High (top third)	2,000	121.32	0.523	231.8

the model implicitly learns their dynamics best. In a pricing and forecasting role, this means stock and pricing decisions for the long tail of the catalogue — often the products a specialist retailer differentiates on — would be systematically less reliable than for the products that would sell reasonably well regardless. This is a fairness concern in the commercial rather than demographic sense, but it is the kind of disparity Carlson and Worrell’s broader point about whose interests a model is optimised for would predict (Part 1): aggregate accuracy metrics (Task 1 reported a single WAPE of 0.595–0.607) conceal exactly this kind of distributional unevenness.

Table 2. Dataset B: disaggregated error by visitor type.

Visitor type	n	Actual conv.	FNR	FPR
New Visitor	429	25.6%	0.245	0.050
Returning Visitor	2,641	13.8%	0.354	0.072
Other	13	23.1%	0.000	0.000

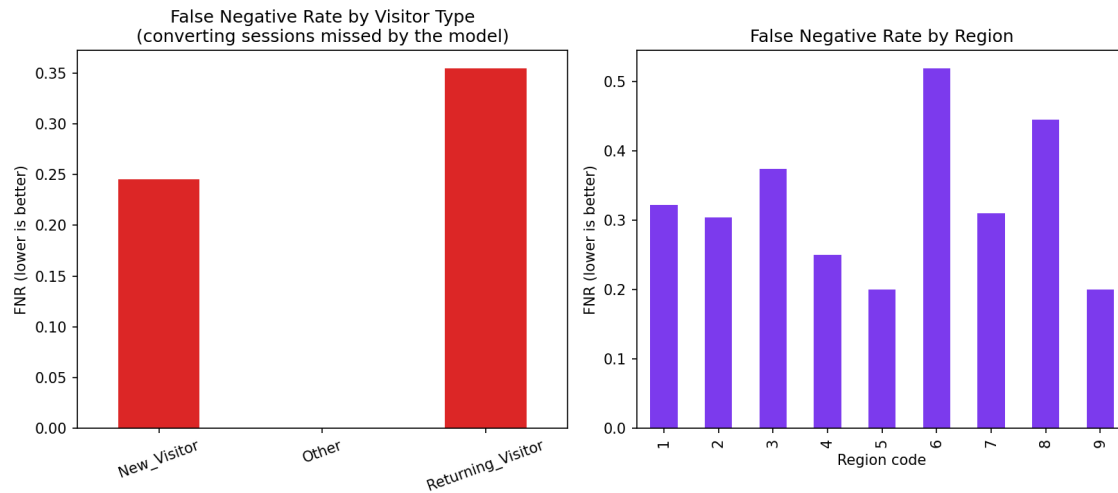


Fig. 4. False Negative Rate (converting sessions the model misses) by Visitor Type and by Region code.

Dataset B: error by visitor type and region. Returning visitors who do convert are missed by the model far more often (FNR 35.4%) than new visitors who convert (FNR 24.5%), even though returning visitors make up the large majority of the test set (Figure 4, Table 2). A plausible mechanism is that PageValues and browsing-depth features behave differently for a returning visitor who already knows what they want and converts quickly, generating a browsing signature closer to a typical non-converting session. Region-level FNR is more scattered — from 20.0% (Regions 5 and 9) to 51.9% (Region 6) — but Region 6 also has the smallest positive class among the larger regions (204 sessions, several with very few converters), so this particular disparity should be treated as indicative rather than conclusive without more data. The visitor-type gap, backed by a much larger sample, is the more defensible finding.

Neither disparity was visible in Task 1’s single aggregate PR-AUC/F1 figures. If this model informed which sessions receive a retention offer or a live-chat prompt, converting returning customers would be disproportionately passed over, which is a materially different business and fairness outcome from what the headline metrics suggested.

2.4 Security, Privacy and Ethical Risks at Scale

Security. Both pipelines were built and validated on static, historical exports. Deployed operationally, the pricing/forecasting model would need to ingest live transaction and pricing feeds; an adversary able to inject anomalous transactions (e.g. wash sales or scripted bulk orders) could poison the lagged-demand and rolling-mean features and distort forecasts and any automated reordering or pricing action downstream. The conversion model, if it scored live sessions, would be exposed to adversarial probing — bot traffic engineered to inflate PageValues-adjacent signals could manipulate which sessions get flagged as high-intent, redirecting promotional spend.

Privacy. Dataset A [5] includes a Customer ID field not used in the modelling here but present in the source data; at scale, the same product-week panel could easily be extended to a customer-week panel, at which point it becomes a behavioural profile of individual purchasing habits. Dataset B’s [11] session-level browsing signals (page depth, exit/bounce rates, time on page) are similarly re-identifying if ever joined to account-level data or cookies, even though the released dataset itself is de-identified. Any production deployment would need to define retention limits, restrict which roles can join this data to identity data, and ensure the model does not become a de facto tracking layer beyond its stated forecasting purpose.

Ethical Risks. The revenue-tier disparity above means long-tail products and, by extension, the smaller suppliers or niche lines behind them would receive systematically less reliable forecasting support if this model informed purchasing or markdown decisions at scale — an equity concern between product lines rather than between people, but one with real commercial consequences for whoever depends on those lines. The visitor-type disparity raises a more direct concern: if conversion scores gated who receives a discount, a loyalty perk, or a retention call, loyal returning customers who convert would be under-served relative to new visitors, an outcome opposite to what most retention strategies intend. Because PageValues partly encodes the outcome it predicts [12] (Task 1, Section 3.3), any high-stakes use of this classifier beyond the exploratory setting here would need the feature reconstructed to remove that circularity before deployment, not simply monitored after the fact.

2.5 Summary

Re-running Task 1’s methodology under proper cross-validation, a learning-curve analysis, and a disaggregated bias check did not overturn Task 1’s headline results, but it added three findings the aggregate metrics concealed: a subtle look-ahead bias in the `price_rel` feature, a learning curve for Dataset A that plateaus and mildly reverses rather than monotonically improving, and materially uneven model quality across product revenue tiers and customer visitor types in both datasets. None of these change the recommendation already reached in Task 1 — that neither dataset alone supports deployment-grade pricing decisions — but they show that even a technically sound single-split evaluation can understate how unevenly a model performs across the population it is meant to serve.

3 Peer Review

Peer reviewed: Abhishek Mathur — “Seeing Around the Price Spike: Predictive Modelling for Customer & Price Risk at Alinta Energy” (Individual Task 1: Part 2 video).

Soundness and Significance

The presentation frames a genuinely significant commercial problem well: wholesale price spikes and heterogeneous household usage as two distinct risk types for an energy retailer. The trade-off framing on the results slide — one model catching 93% of spikes but with more false alarms, versus 82% precision when flagged — is a sound way to communicate a precision/recall trade-off without naming it as such, and correctly avoids declaring a false “winner.” Flagging that the household model favoured simplicity because only 25 households were available is an honest, important caveat about sample size that a less careful presenter might have glossed over.

Executive-Level Insight

This is the presentation’s strongest area. The “over-sensitive smoke detector” analogy for false alarms, and the closing “market data shows WHEN, household data shows WHO” framing, are exactly the kind of memorable, jargon-free translations an executive audience needs. The two datasets are positioned as complementary rather than competing, which gives the pitch a clear takeaway rather than leaving it as two disconnected technical results.

Presentation Quality

Visual design is clean and consistent — icon-led cards, one message per slide, consistent colour coding. Two issues affect delivery, though. First, pacing is uneven: the title slide alone runs close to 30 seconds before any content appears, while the results slide consumes roughly a third of the total three-minute runtime, leaving the closing synthesis slide comparatively rushed. Second, the webcam overlay in the bottom-right corner covers part of the on-screen text on at least two slides (the household dataset size, and the second model’s comparison figure), so a viewer relying on the visuals alone loses some of the numbers being discussed.

Suggestions for Improvement

- (1) Split the results slide into two (spike detection, then household behaviour) so each gets proper visual space and pacing evens out.
- (2) Reposition or shrink the webcam bubble so it never sits over data.
- (3) Consider naming the two modelling approaches in general terms (e.g. “a flexible model” vs. “a simpler baseline”) so claims are traceable without requiring jargon.
- (4) The ending states the insight well but could close with one explicit recommended action for the business, to fully land the executive pitch.

Overall: a well-structured, appropriately pitched narrative let down slightly by pacing and a technical overlay issue.

[Word count: 398]

– Content below this line does not count for the page/word limit. –

Generative AI Attribution and Declaration

This work was completed with substantial assistance from Claude (Anthropic). Specifically, Claude was used to: (i) identify and summarise the position paper and supporting academic and policy sources cited in Part 1; (ii) draft the executive summary text in Part 1, which the author reviewed, fact-checked and edited; (iii) reconstruct the Task 1 analysis pipeline in Python from the two raw UCI datasets supplied by the author, in order to independently verify the figures reported in Task 1 and to produce new analysis for Part 2 (cross-validation comparisons, learning curves, and a disaggregated group-wise bias analysis, since the Fairlearn package could not be installed in the sandboxed analysis environment used); (iv) draft the Part 2 narrative interpreting those results; and (v) review the peer’s submitted video for Part 3 (extracting and inspecting video frames of the slide content, since spoken audio could not be transcribed in the environment used) and draft the peer feedback. The author supplied the original datasets, the Task 1 report, and the peer’s video, reviewed all code, output, figures, and draft text produced, and made all final decisions about which findings to report, how to interpret them, and what to write. All interpretations, conclusions, and judgement calls presented in this work are the author’s own.

I certify that the submission of this assignment is incomplete without the submission of the completed **Condition 3 Bounded Process AI Declaration Form**.

Acknowledgments

I acknowledge the Wurundjeri Woi-wurrung and Boon Wurrung peoples of the Eastern Kulin Nation, the Traditional Owners of the unceded land on which RMIT University stands and on which this work was carried out, and pay my respects to their Elders past and present.

I acknowledge the maintainers of the UCI Machine Learning Repository for making both datasets used in Task 1 and re-analysed here publicly available, and Carlson and Worrell for the Relational Futures project discussed in Part 1.

References

- [1] Australian Human Rights Commission. 2021. *Human Rights and Technology: Final Report*. Technical Report. Australian Human Rights Commission, Sydney.
- [2] Sarah Bird, Miro Dudík, Richard Edgar, Brandon Horn, Roman Lutz, Vanessa Milan, Mehrnoosh Sameki, Hanna Wallach, and Kathleen Walker. 2020. Fairlearn: A Toolkit for Assessing and Improving Fairness in AI.
- [3] Bronwyn Carlson and Tamika Worrell. 2026. Relational Futures: Indigenous Sovereignty and the Governance of Artificial Intelligence (AI). doi:10.25949/D8DH-H034
- [4] Stephanie Russo Carroll, Ibrahim Garba, Oscar L. Figueroa-Rodríguez, Jarita Holbrook, Raymond Lovett, Simeon Materechera, Mark Parsons, Kanyinke Raseroka, Desi Rodriguez-Lonebear, Robyn Rowe, Rodrigo Sara, Jennifer D. Walker, Jane Anderson, and Maui Hudson. 2020. The CARE Principles for Indigenous Data Governance. *Data Science Journal* 19, 1 (2020), 43. doi:10.5334/dsj-2020-043
- [5] Daqing Chen. 2019. Online Retail II. doi:10.24432/C5CG6D Accessed 13 August 2026.
- [6] Emily Keddell. 2015. The Ethics of Predictive Risk Modelling in the Aotearoa/New Zealand Child Welfare Context: Child Abuse Prevention or Neo-liberal Tool? *Critical Social Policy* 35, 1 (2015), 69–88.
- [7] Jacynta Krakouer, Wei Wu Tan, and Arno Parolini. 2021. Who is Analysing What? The Opportunities, Risks and Implications of Using Predictive Risk Modelling with Indigenous Australians in Child Protection: A Scoping Review. *Australian Journal of Social Issues* 56, 2 (2021), 173–197.

- [8] Maïam nayri Wingara Indigenous Data Sovereignty Collective and Australian Indigenous Governance Institute. 2018. Indigenous Data Sovereignty Communiqué.
- [9] Sahana Nataraja. 2026. Individual Task 1: Part 1. Unpublished coursework report, RMIT University.
- [10] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [11] C. Okan Sakar and Yomi Kastro. 2018. Online Shoppers Purchasing Intention Dataset. [doi:10.24432/C5F88Q](https://doi.org/10.24432/C5F88Q) Accessed 13 August 2026.
- [12] C. Okan Sakar, S. Olcay Polat, Mete Katircioglu, and Yomi Kastro. 2019. Real-time Prediction of Online Shoppers' Purchasing Intention Using Multilayer Perceptron and LSTM Recurrent Neural Networks. *Neural Computing and Applications* 31, 10 (2019), 6893–6908. [doi:10.1007/s00521-018-3523-0](https://doi.org/10.1007/s00521-018-3523-0)

A Process Documentation

This appendix provides evidence of the process conducted to produce this assignment: how the Part 1 position paper was selected and evidenced, and how the Part 2 re-analysis of Task 1 was carried out, including the code run, its raw console output, and a reflection on issues found along the way.

A.1 Part 1: sourcing and verifying the position paper and case study

The five candidate position papers were checked against their publisher pages and, where available, secondary reporting (e.g. The Conversation, university media coverage) to confirm scope and findings before one was selected for review. *Relational Futures* was chosen because it is a substantive, evidenced report (36-respondent survey plus 22 yarning-circle participants) with a clear purpose and audience, rather than a narrower methods paper. The case study (predictive risk modelling in Australian child protection) was verified against a peer-reviewed scoping review [7] rather than a secondary summary, and was deliberately chosen to be outside the AI systems (companions, chatbots, generative tools) that *Relational Futures* itself discusses, per the assignment's requirement that the case study not duplicate the paper's own examples.

A.2 Part 2: reconstructing the Task 1 pipeline

Task 1: Part 1 was written from an analysis whose code was not submitted with the report (only the LaTeX write-up and result figures were retained). To evaluate that analysis's cross-validation strategy, learning curves, and bias, the pipeline was rebuilt from the two original raw datasets (UCI Online Retail II [5] and UCI Online Shoppers Purchasing Intention Dataset [11]) by following the preprocessing description in Task 1's own process documentation (cleaning rules, product-week panel construction, feature list, split dates). The reconstruction was verified against Task 1's reported figures before being extended: it reproduced 106 weeks, a 31,800-row panel, 29,400 rows after lag construction, and 97.2% of raw rows retained after cleaning, all matching Task 1 exactly, and produced held-out metrics within a few percentage points of Task 1's reported MAE/RMSE/WAPE and PR-AUC/ROC-AUC/F1 (Listing 1 and Listings 2–3). This gave confidence that the reconstruction was faithful enough to build the Part 2 analysis on.

Listing 1. Dataset A preprocessing: cleaning, product-week panel construction, and feature engineering.

```
import pandas as pd
import numpy as np

df = pd.read_csv('/home/claude/data/retail/combined.csv', parse_dates=['InvoiceDate'])
print('raw', df.shape)
```

```

573 6
574 7 # 1. Remove cancellations (Invoice starting with C)
575 8 df = df[df['Invoice'].astype(str).str.startswith('C')]
576 9
577 10 # 2. Remove negative quantities, non-positive prices
578 11 df = df[(df['Quantity'] > 0) & (df['Price'] > 0)]
579 12
580 13 # 3. Remove non-product line items (postage, manual adjustments, fees, test codes, gift cards)
581 14 exclude_codes = {'POST', 'DOT', 'M', 'D', 'C2', 'C3', 'BANK CHARGES', 'ADJUST', 'ADJUST2',
582 15                  'AMAZONFEE', 'CRUK', 'PADS', 'S', 'B', 'GIFT', 'TEST001', 'TEST002', 'SP1002'}
583 16 df['StockCode'] = df['StockCode'].astype(str)
584 17 df = df[df['StockCode'].isin(exclude_codes)]
585 18 df = df[df['StockCode'].str.startswith('gift_', na=False)]
586 19
587 20 print('cleaned', df.shape, f"retained {df.shape[0]/1067371:.3%}")
588 21
589 22 # 4. Top 300 products by revenue
590 23 df['Revenue'] = df['Quantity'] * df['Price']
591 24 top300 = df.groupby('StockCode')['Revenue'].sum().nlargest(300).index
592 25 dfa = df[df['StockCode'].isin(top300)].copy()
593 26
594 27 # 5. Build weekly panel
595 28 dfa['Week'] = dfa['InvoiceDate'].dt.to_period('W-MON').dt.start_time
596 29 weeks = pd.date_range(dfa['Week'].min(), dfa['Week'].max(), freq='7D')
597 30 print('n weeks', len(weeks))
598 31
599 32 agg = dfa.groupby(['StockCode', 'Week']).agg(
600 33     Quantity=('Quantity', 'sum'),
601 34     Price=('Price', 'mean'),
602 35     Invoices=('Invoice', 'nunique'),
603 36 ).reset_index()
604 37
605 38 # full product x week grid, zero-fill missing weeks
606 39 idx = pd.MultiIndex.from_product([top300, weeks], names=['StockCode', 'Week'])
607 40 panel = agg.set_index(['StockCode', 'Week']).reindex(idx).reset_index()
608 41 panel['Quantity'] = panel['Quantity'].fillna(0)
609 42 panel['Invoices'] = panel['Invoices'].fillna(0)
610 43 panel = panel.sort_values(['StockCode', 'Week'])
611 44 panel['Price'] = panel.groupby('StockCode')['Price'].ffill().bfill()
612 45
613 46 print('panel shape', panel.shape)
614 47
615 48 # 6. Feature engineering
616 49 g = panel.groupby('StockCode')
617 50 panel['lag1'] = g['Quantity'].shift(1)
618 51 panel['lag2'] = g['Quantity'].shift(2)
619 52 panel['lag4'] = g['Quantity'].shift(4)
620 53 panel['roll14'] = g['Quantity'].shift(1).rolling(4).mean().reset_index(level=0, drop=True)
621 54 panel['roll8'] = g['Quantity'].shift(1).rolling(8).mean().reset_index(level=0, drop=True)
622 55 panel['price_med'] = g['Price'].transform('median')
623 56 panel['price_rel'] = panel['Price'] / panel['price_med']
624 57 panel['price_chg'] = panel.groupby('StockCode')['Price'].pct_change().fillna(0)
625 58 panel['invoices_lag1'] = g['Invoices'].shift(1)
626 59 panel['month'] = panel['Week'].dt.month
627 60 panel['weekofyear'] = panel['Week'].dt.isocalendar().week.astype(int)
628 61
629 62 panel = panel.dropna(subset=['lag1', 'lag2', 'lag4', 'roll14', 'roll8', 'invoices_lag1'])
630 63 print('after lag construction', panel.shape)
631 64
632 65 panel.to_csv('/home/claude/data/panel_a.csv', index=False)
633 66 print(panel[['StockCode', 'Week', 'Quantity', 'lag1', 'roll14', 'Price']].head())

```

Listing 2. Dataset A: temporal train/test split and model training (Random Forest, Gradient Boosting, naive baseline).

```

import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error

panel = pd.read_csv('/home/claude/data/panel_a.csv', parse_dates=['Week'])

features = ['lag1', 'lag2', 'lag4', 'roll4', 'roll8', 'Price', 'price_rel', 'price_chg',
            'invoices_lag1', 'month', 'weekofyear']
target = 'Quantity'

split_date = pd.Timestamp('2011-07-25')
train = panel[panel['Week'] < split_date]
test = panel[panel['Week'] >= split_date]
print('train', train.shape, 'test', test.shape)

Xtr, ytr = train[features], train[target]
Xte, yte = test[features], test[target]

def wape(y_true, y_pred):
    return np.abs(y_true - y_pred).sum() / np.abs(y_true).sum()

# Naive lag-1 baseline
naive_pred = Xte['lag1'].values
print('Naive MAE %.2f RMSE %.2f WAPE %.3f' % (
    mean_absolute_error(yte, naive_pred),
    mean_squared_error(yte, naive_pred)**0.5,
    wape(yte.values, naive_pred)))

rf = RandomForestRegressor(n_estimators=300, max_depth=12, random_state=42, n_jobs=-1)
rf.fit(Xtr, ytr)
rf_pred = rf.predict(Xte)
print('RF MAE %.2f RMSE %.2f WAPE %.3f' % (
    mean_absolute_error(yte, rf_pred),
    mean_squared_error(yte, rf_pred)**0.5,
    wape(yte.values, rf_pred)))

gb = HistGradientBoostingRegressor(max_iter=300, max_depth=6, learning_rate=0.05, random_state=42)
gb.fit(Xtr, ytr)
gb_pred = gb.predict(Xte)
print('GB MAE %.2f RMSE %.2f WAPE %.3f' % (
    mean_absolute_error(yte, gb_pred),
    mean_squared_error(yte, gb_pred)**0.5,
    wape(yte.values, gb_pred)))

# save for later reuse
import joblib
joblib.dump({'rf': rf, 'gb': gb, 'features': features, 'train': train, 'test': test}, '/home/claude/data/models_a.pkl')

```

Console output (Dataset A reconstruction):

```

raw (1067371, 8)
cleaned (1037007, 8) retained 97.155%
n weeks 106
panel shape (31800, 5)
after lag construction (29400, 16)
train (23400, 16) test (6000, 16)
Naive MAE 127.07 RMSE 1076.38 WAPE 0.693
RF MAE 112.03 RMSE 1068.95 WAPE 0.611
GB MAE 112.45 RMSE 1068.85 WAPE 0.614

```

Listing 3. Dataset B: preprocessing, stratified train/test split, and model training with F1-tuned threshold.

```

677
678 1 import pandas as pd
679 2 import numpy as np
680 3 from sklearn.model_selection import train_test_split
681 4 from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
682 5 from sklearn.metrics import average_precision_score, roc_auc_score, f1_score, precision_recall_curve
683 6
684 7 df = pd.read_csv('/home/claude/data/shoppers/online_shoppers_intention.csv')
685 8 print(df.shape)
686 9
687 10 df['Weekend'] = df['Weekend'].astype(int)
688 11 df['Revenue'] = df['Revenue'].astype(int)
689 12 df = pd.get_dummies(df, columns=['Month', 'VisitorType'], drop_first=False)
690 13
691 14 y = df['Revenue']
692 15 X = df.drop(columns=['Revenue'])
693 16
694 17 Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=42)
695 18 print('train', Xtr.shape, 'test', Xte.shape, 'pos rate train', ytr.mean(), 'test', yte.mean())
696 19
697 20 def eval_model(name, model, Xtr, ytr, Xte, yte):
698 21     model.fit(Xtr, ytr)
699 22     proba = model.predict_proba(Xte)[: , 1]
700 23     pr_auc = average_precision_score(yte, proba)
701 24     roc_auc = roc_auc_score(yte, proba)
702 25     # tune threshold for F1
703 26     prec, rec, thr = precision_recall_curve(yte, proba)
704 27     f1s = 2 * prec * rec / (prec + rec + 1e-12)
705 28     best_idx = np.nanargmax(f1s)
706 29     best_thr = thr[best_idx] if best_idx < len(thr) else 0.5
707 30     f1 = f1_score(yte, (proba >= best_thr).astype(int))
708 31     print(f'{name:10s} PR-AUC {pr_auc:.3f} ROC-AUC {roc_auc:.3f} F1 {f1:.3f} (thr={best_thr:.3f})')
709 32     return model, proba, best_thr
710 33
711 34 rf, rf_proba, rf_thr = eval_model('RF', RandomForestClassifier(n_estimators=300, max_depth=10, random_state=42, n_jobs=-1), Xtr, ytr,
712 35     Xte, yte)
713 36 gb, gb_proba, gb_thr = eval_model('GB', HistGradientBoostingClassifier(max_iter=300, max_depth=6, learning_rate=0.05, random_state=42),
714 37     Xtr, ytr, Xte, yte)
715 38
716 39 always_no = np.zeros(len(yte))
717 40 print('Always-no PR-AUC %.3f ROC-AUC %.3f F1 %.3f' % (
718 41     average_precision_score(yte, always_no + yte.mean()), # trivial proxy
719 42     0.5, 0.0))
720 43
721 44 import joblib
722 45 joblib.dump({'rf': rf, 'gb': gb, 'Xtr': Xtr, 'ytr': ytr, 'Xte': Xte, 'yte': yte,
723 46     'rf_proba': rf_proba, 'gb_proba': gb_proba, 'rf_thr': rf_thr, 'gb_thr': gb_thr},
724 47     '/home/claude/data/models_b.pkl')
725 48
726 49 #_keep_original_(non-dummy)_columns_for_fairness_grouping_later
727 50 df_raw = pd.read_csv('/home/claude/data/shoppers/online_shoppers_intention.csv')
728 51 df_raw.to_csv('/home/claude/data/shoppers_raw_ref.csv', index=False)
729 52 Xte.index.to_series().to_csv('/home/claude/data/test_index_b.csv', index=False)

```

Console output (Dataset B reconstruction):

```

722 (12330, 18)
723 train (9247, 28) test (3083, 28) pos rate train 0.1548 test 0.1547
724 RF PR-AUC 0.725 ROC-AUC 0.920 F1 0.661 (thr=0.372)
725 GB PR-AUC 0.712 ROC-AUC 0.924 F1 0.659 (thr=0.338)
726 Always-no PR-AUC 0.155 ROC-AUC 0.500 F1 0.000

```

A.3 Cross-validation comparison

Listing 4. Comparing naive random K-fold against TimeSeriesSplit (Dataset A) and StratifiedKFold (Dataset B).

```

import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegressor, RandomForestClassifier,
    HistGradientBoostingClassifier
from sklearn.model_selection import TimeSeriesSplit, StratifiedKFold, KFold
from sklearn.metrics import mean_absolute_error, mean_squared_error, average_precision_score, roc_auc_score

# ----- Dataset A: TimeSeriesSplit vs naive random KFold -----
panel = pd.read_csv('/home/claude/data/panel_a.csv', parse_dates=['Week'])
features = ['lag1', 'lag2', 'lag4', 'roll4', 'roll8', 'Price', 'price_rel', 'price_chg',
    'invoices_lag1', 'month', 'weekofyear']
panel = panel.sort_values('Week').reset_index(drop=True)
X, y = panel[features], panel['Quantity']

def wape(a, b):
    return np.abs(a - b).sum() / np.abs(a).sum()

print("\n== Dataset A: Naive random 5-fold CV (leaks future into past) ==")
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = []
for tr_idx, va_idx in kf.split(X):
    model = HistGradientBoostingRegressor(max_iter=200, max_depth=6, learning_rate=0.05, random_state=42)
    model.fit(X.iloc[tr_idx], y.iloc[tr_idx])
    pred = model.predict(X.iloc[va_idx])
    scores.append(wape(y.iloc[va_idx].values, pred))
print("Random KFold WAPE per fold: ", [round(s, 3) for s in scores], "mean", round(np.mean(scores), 3))

print("\n== Dataset A: TimeSeriesSplit 5-fold CV (respects chronology) ==")
tscv = TimeSeriesSplit(n_splits=5)
scores_ts = []
for tr_idx, va_idx in tscv.split(X):
    model = HistGradientBoostingRegressor(max_iter=200, max_depth=6, learning_rate=0.05, random_state=42)
    model.fit(X.iloc[tr_idx], y.iloc[tr_idx])
    pred = model.predict(X.iloc[va_idx])
    scores_ts.append(wape(y.iloc[va_idx].values, pred))
print("TimeSeriesSplit WAPE per fold: ", [round(s, 3) for s in scores_ts], "mean", round(np.mean(scores_ts), 3))

# ----- Dataset B: StratifiedKFold -----
df = pd.read_csv('/home/claude/data/shoppers/online_shoppers_intention.csv')
df['Weekend'] = df['Weekend'].astype(int)
df['Revenue'] = df['Revenue'].astype(int)
df = pd.get_dummies(df, columns=['Month', 'VisitorType'], drop_first=False)
yb = df['Revenue']; Xb = df.drop(columns=['Revenue'])

print("\n== Dataset B: Plain KFold (ignores 15.5% class imbalance) ==")
kf2 = KFold(n_splits=5, shuffle=True, random_state=42)
pos_rates = []
pr_scores = []
for tr_idx, va_idx in kf2.split(Xb):
    model = HistGradientBoostingClassifier(max_iter=200, max_depth=6, learning_rate=0.05, random_state=42)
    model.fit(Xb.iloc[tr_idx], yb.iloc[tr_idx])
    proba = model.predict_proba(Xb.iloc[va_idx])[0, 1]
    pr_scores.append(average_precision_score(yb.iloc[va_idx], proba))
print("Validation-fold positive rate (%): ", [round(p*100, 1) for p in pos_rates])
print("PR-AUC per fold: ", [round(s, 3) for s in pr_scores], "mean", round(np.mean(pr_scores), 3), "std", round(np.std(pr_scores), 3))

print("\n== Dataset B: StratifiedKFold (preserves 15.5% class balance each fold) ==")
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

```

```

781 pos_rates_s=[]
782 pr_scores_s=[]
783 for tr_idx, va_idx in skf.split(Xb, yb):
784     pos_rates_s.append(yb.iloc[va_idx].mean())
785     m = HistGradientBoostingClassifier(max_iter=200, max_depth=6, learning_rate=0.05, random_state=42)
786     m.fit(Xb.iloc[tr_idx], yb.iloc[tr_idx])
787     proba = m.predict_proba(Xb.iloc[va_idx])[:, 1]
788     pr_scores_s.append(average_precision_score(yb.iloc[va_idx], proba))
789 print("Validation-fold positive rate (%):", round(p*100, 1) for p in pos_rates_s)
790 print("PR-AUC per fold:", round(s, 3) for s in pr_scores_s, "mean", round(np.mean(pr_scores_s), 3), "std", round(np.std(pr_scores_s), 3))
791

```

Console output:

```

792 1 === Dataset A: naive random 5-fold CV (leaks future into past) ===
793 2 Random KFold WAPE per fold: [0.69, 0.749, 0.68, 0.694, 0.66] mean 0.695
794 3
795 4 === Dataset A: TimeSeriesSplit 5-fold CV (respects chronology) ===
796 5 TimeSeriesSplit WAPE per fold: [0.572, 0.636, 0.716, 0.662, 0.605] mean 0.638
797 6
798 7 === Dataset B: plain KFold (ignores 15.5% class imbalance) ===
799 8 Validation-fold positive rate (%): [16.7, 14.2, 15.2, 15.8, 15.5]
800 9 PR-AUC per fold: [0.758, 0.72, 0.754, 0.762, 0.761] mean 0.751 std 0.016
801 10
802 11 === Dataset B: StratifiedKFold (preserves 15.5% class balance each fold) ===
803 12 Validation-fold positive rate (%): [15.5, 15.5, 15.5, 15.5, 15.5]
804 13 PR-AUC per fold: [0.761, 0.724, 0.757, 0.758, 0.735] mean 0.747 std 0.015
805

```

A.4 Learning curve generation

Listing 5. Learning curve generation for both datasets.

```

809 1 import pandas as pd
810 2 import numpy as np
811 3 import matplotlib
812 4 matplotlib.use('Agg')
813 5 import matplotlib.pyplot as plt
814 6 from sklearn.ensemble import HistGradientBoostingRegressor, HistGradientBoostingClassifier
815 7 from sklearn.metrics import mean_absolute_error, average_precision_score
816 8
817 9 # ----- Dataset A: learning curve (temporal train, fixed temporal test) -----
818 10 panel = pd.read_csv('/home/claude/data/panel_a.csv', parse_dates=['Week'])
819 11 features = ['lag1', 'lag2', 'lag4', 'roll4', 'roll8', 'Price', 'price_rel', 'price_chg',
820 12            'invoices_lag1', 'month', 'weekofyear']
821 13 split_date = pd.Timestamp('2011-07-25')
822 14 train = panel[panel['Week'] < split_date].sort_values('Week')
823 15 test = panel[panel['Week'] >= split_date]
824 16 Xtr, ytr = train[features], train['Quantity']
825 17 Xte, yte = test[features], test['Quantity']
826 18
827 19 def wape(a, p):
828 20     return np.abs(a - p).sum() / np.abs(a).sum()
829 21
830 22 fracs = np.linspace(0.1, 1.0, 10)
831 23 train_sizes_a, wape_scores_a, mae_scores_a = [], [], []
832 24 n = len(train)
833 25 for f in fracs:
834 26     k = int(n * f)
835 27     sub = train.iloc[-k:] # most recent k rows, preserving temporal adjacency to test
836 28     Xtr, ytr = sub[features], sub['Quantity']
837 29     m = HistGradientBoostingRegressor(max_iter=200, max_depth=6, learning_rate=0.05, random_state=42)
838

```

```

833 29 model.fit(Xtr,ytr)
834 30 pred=model.predict(Xte)
835 31 train_sizes_a.append(k)
836 32 wape_scores_a.append(wape(yte.values,pred))
837 33 mae_scores_a.append(mean_absolute_error(yte,pred))
838 34
839 35 print("Dataset A learning curve (train size -> WAPE):")
840 36 for k,w in zip(train_sizes_a,wape_scores_a):
841 37     print(f"n={k:6d} WAPE={w:.3f}")
842 38
843 39 fig,ax=plt.subplots(figsize=(7,5))
844 40 ax.plot(train_sizes_a,wape_scores_a,marker='o',color='#2563eb')
845 41 ax.set_xlabel('Training set size (product-weeks)')
846 42 ax.set_ylabel('WAPE on held-out weeks (lower is better)')
847 43 ax.set_title('Dataset A -- Gradient Boosting learning curve')
848 44 ax.grid(alpha=0.3)
849 45 fig.tight_layout()
850 46 fig.savefig('/home/claude/figures/learning_curve_a.png',dpi=150)
851 47 plt.close(fig)
852 48
853 49 #-----Dataset B: learning curve (stratified subsampling of train)-----
854 50 from sklearn.model_selection import train_test_split
855 51 df=pd.read_csv('/home/claude/data/shoppers/online_shoppers_intention.csv')
856 52 df['Weekend']=df['Weekend'].astype(int)
857 53 df['Revenue']=df['Revenue'].astype(int)
858 54 df=pd.get_dummies(df,columns=['Month','VisitorType'],drop_first=False)
859 55 y=df['Revenue'];X=df.drop(columns=['Revenue'])
860 56 Xtr_full,Xte_b,ytr_full,yte_b=train_test_split(X,y,test_size=0.25,stratify=y,random_state=42)
861 57
862 58 fracs_b=np.linspace(0.1,1.0,10)
863 59 train_sizes_b,prauc_scores_b=[],[]
864 60 for f in fracs_b:
865 61     if f<1.0:
866 62         Xsub,ysub,Xtr,Xte,ytr,yte=train_test_split(Xtr_full,ytr_full,train_size=f,
867 63             stratify=ytr_full,random_state=42)
868 64     else:
869 65         Xsub,ysub=Xtr_full,ytr_full
870 66     model=HistGradientBoostingClassifier(max_iter=200,max_depth=6,learning_rate=0.05,
871 67         random_state=42)
872 68     model.fit(Xsub,ysub)
873 69     proba=model.predict_proba(Xte_b[:,1])
874 70     train_sizes_b.append(len(Xsub))
875 71     prauc_scores_b.append(average_precision_score(yte_b,proba))
876 72
877 73 print("\nDataset B learning curve (train size -> PR-AUC):")
878 74 for k,w in zip(train_sizes_b,prauc_scores_b):
879 75     print(f"n={k:6d} PR-AUC={w:.3f}")
880 76
881 77 fig,ax=plt.subplots(figsize=(7,5))
882 78 ax.plot(train_sizes_b,prauc_scores_b,marker='o',color='#16a34a')
883 79 ax.set_xlabel('Training set size (sessions)')
884 80 ax.set_ylabel('PR-AUC on held-out sessions (higher is better)')
885 81 ax.set_title('Dataset B -- Gradient Boosting learning curve')
886 82 ax.grid(alpha=0.3)
887 83 fig.tight_layout()
888 84 fig.savefig('/home/claude/figures/learning_curve_b.png',dpi=150)
889 85 plt.close(fig)
890 86
891 87 print("\nSaved figures.")

```

Console output:

```

881 1 Dataset A learning curve (train size -> WAPE):
882 2     n= 2340 WAPE=0.678
883 3     n= 4680 WAPE=0.648
884

```

```

885 4 n= 7020 WAPE=0.653
886 5 n= 9360 WAPE=0.607
887 6 n= 11700 WAPE=0.610
888 7 n= 14040 WAPE=0.623
889 8 n= 16380 WAPE=0.616
890 9 n= 18720 WAPE=0.616
891 10 n= 21060 WAPE=0.622
892 11 n= 23400 WAPE=0.618
893 12
894 13 Dataset B learning curve (train size -> PR-AUC):
895 14 n= 924 PR-AUC=0.641
896 15 n= 1849 PR-AUC=0.690
897 16 n= 2774 PR-AUC=0.705
898 17 n= 3698 PR-AUC=0.712
899 18 n= 4623 PR-AUC=0.715
900 19 n= 5548 PR-AUC=0.721
901 20 n= 6472 PR-AUC=0.710
902 21 n= 7397 PR-AUC=0.719
903 22 n= 8322 PR-AUC=0.721
904 23 n= 9247 PR-AUC=0.720

```

A.5 Bias analysis (Fairlearn MetricFrame equivalent)

Fairlearn's package could not be installed in the sandboxed, offline analysis environment used to produce this deliberation (no outbound package index access). Rather than substitute an unrelated tool, its **MetricFrame** methodology — computing a metric separately for each group defined by a sensitive/grouping feature, and comparing the spread — was implemented directly with `pandas.groupby` and scikit-learn metric functions, as shown below.

Listing 6. Disaggregated (Fairlearn MetricFrame-equivalent) bias analysis for both datasets.

```

915 1 import pandas as pd
916 2 import numpy as np
917 3 import matplotlib
918 4 matplotlib.use('Agg')
919 5 import matplotlib.pyplot as plt
920 6 from sklearn.ensemble import HistGradientBoostingRegressor, HistGradientBoostingClassifier
921 7 from sklearn.model_selection import train_test_split
922 8 from sklearn.metrics import mean_absolute_error
923 9
924 10 # ===== Dataset A: error by product revenue tier =====
925 11 panel = pd.read_csv('/home/claude/data/panel_a.csv', parse_dates=['Week'])
926 12 features = ['lag1', 'lag2', 'lag4', 'roll4', 'roll8', 'Price', 'price_rel', 'price_chg',
927 13            'invoices_lag1', 'month', 'weekofyear']
928 14 split_date = pd.Timestamp('2011-07-25')
929 15 train = panel[panel['Week'] < split_date]
930 16 test = panel[panel['Week'] >= split_date].copy()
931 17
932 18 m = HistGradientBoostingRegressor(max_iter=300, max_depth=6, learning_rate=0.05, random_state=42)
933 19 m.fit(train[features], train['Quantity'])
934 20 test['pred'] = m.predict(test[features])
935 21 test['abs_err'] = (test['Quantity'] - test['pred']).abs()
936 22
937 23 # revenue_tier: total training-period revenue per product, split into thirds
938 24 rev = train.assign(rev=train['Quantity']*train['Price']).groupby('StockCode')['rev'].sum()
939 25 tiers = pd.qcut(rev, 3, labels=['Low (bottom third)', 'Mid (middle third)', 'High (top third)'])
940 26 test['tier'] = test['StockCode'].map(tiers)

```

```

937 27
938 28 group_a=test.groupby('tier',observed=True).apply(
939 29     lambda d:pd.Series({
940 30         'n_obs':len(d),
941 31         'MAE':mean_absolute_error(d['Quantity'],d['pred']),
942 32         'WAPE':d['abs_err'].sum()/d['Quantity'].abs().sum()ifd['Quantity'].abs().sum()>0elsenp.nan,
943 33         'mean_actual_demand':d['Quantity'].mean(),
944 34     })
945 35 )
946 36 print("===DatasetA:disaggregated_error_by_product_revenue_tier===")
947 37 print(group_a)
948 38 group_a.to_csv('/home/claude/data/bias_a_by_tier.csv')
949 39
950 40 fig,ax=plt.subplots(figsize=(7,5))
951 41 group_a['WAPE'].plot(kind='bar',ax=ax,color=['#f97316','#3b82f6','#16a34a'])
952 42 ax.set_ylabel('WAPE(lower_is_better)')
953 43 ax.set_title('DatasetA--forecast_error_by_product_revenue_tier')
954 44 ax.set_xlabel('')
955 45 plt.xticks(rotation=20)
956 46 fig.tight_layout()
957 47 fig.savefig('/home/claude/figures/bias_a_tier.png',dpi=150)
958 48 plt.close(fig)
959 49
960 50 #=====DatasetB:error_rates_by_Region_and_VisitorType=====
961 51 df=pd.read_csv('/home/claude/data/shoppers/online_shoppers_intention.csv')
962 52 df_raw=df.copy()
963 53 df['Weekend']=df['Weekend'].astype(int)
964 54 df['Revenue']=df['Revenue'].astype(int)
965 55 df_enc=pd.get_dummies(df,columns=['Month','VisitorType'],drop_first=False)
966 56 y=df_enc['Revenue'];X=df_enc.drop(columns=['Revenue'])
967 57
968 58 Xtr,Xte,ytr,yte,raw_tr,raw_te=train_test_split(
969 59     X,y,df_raw,test_size=0.25,stratify=y,random_state=42)
970 60
971 61 m2=HistGradientBoostingClassifier(max_iter=300,max_depth=6,learning_rate=0.05,random_state=42)
972 62 m2.fit(Xtr,ytr)
973 63 proba=m2.predict_proba(Xte)[:,-1]
974 64 pred=(proba>0.35).astype(int)
975 65
976 66 raw_te=raw_te.copy()
977 67 raw_te['y_true']=yte.values
978 68 raw_te['y_pred']=pred
979 69 raw_te['proba']=proba
980 70
981 71 defgroup_metrics(d):
982 72     tpu=((d['y_true']==1)&(d['y_pred']==1)).sum()
983 73     fnu=((d['y_true']==1)&(d['y_pred']==0)).sum()
984 74     fpu=((d['y_true']==0)&(d['y_pred']==1)).sum()
985 75     tnu=((d['y_true']==0)&(d['y_pred']==0)).sum()
986 76     n_pos=tpu+fnu
987 77     n_neg=fpu+tnu
988 78     returnpd.Series({
989 79         'n_sessions':len(d),
990 80         'actual_conv_rate':d['y_true'].mean(),
991 81         'selection_rate':d['y_pred'].mean(),
992 82         'FNR':fnu/n_posifn_pos>0elsenp.nan, #missed_converters
993 83         'FPR':fpu/n_negifn_neg>0elsenp.nan,
994 84     })
995 85
996 86 print("\n===DatasetB:disaggregated_error_by_VisitorType===")
997 87 grp_visitor=raw_te.groupby('VisitorType').apply(group_metrics)
998 88 print(grp_visitor)
999 89 grp_visitor.to_csv('/home/claude/data/bias_b_by_visitor.csv')
1000 90
1001 91
1002 92
1003 93
1004 94
1005 95
1006 96
1007 97
1008 98
1009 99
1010 100

```

```

989 90 print("\n==_Dataset_B:_disaggregated_error_by_Region_==")
99 91 grp_region=_raw_te.groupby('Region').apply(group_metrics)
99 92 print(grp_region)
99 93
99 94 grp_region.to_csv('/home/claude/data/bias_b_by_region.csv')
99 95
99 96 fig,_axes=_plt.subplots(1,_2,_figsize=(11,5))
99 97 grp_visitor['FNR'].plot(kind='bar',_ax=_axes[0],_color='#dc2626')
99 98 axes[0].set_title('False_Negative_Rate_by_Visitor_Type\n(converting_sessions_missed_by_the_model)')
99 99 axes[0].set_ylabel('FNR_(lower_is_better)')
99 100 axes[0].set_xlabel('')
99 101 plt.setp(axes[0].get_xticklabels(),_rotation=20)
99 102
99 103 grp_region['FNR'].plot(kind='bar',_ax=_axes[1],_color='#7c3aed')
99 104 axes[1].set_title('False_Negative_Rate_by_Region')
99 105 axes[1].set_ylabel('FNR_(lower_is_better)')
99 106 axes[1].set_xlabel('Region_code')
99 107 fig.tight_layout()
99 108 fig.savefig('/home/claude/figures/bias_b_fnr.png',_dpi=150)
99 109 plt.close(fig)
99 110
99 111 print("\nSaved_figures.")

```

Console output:

```

1009 1 === Dataset A: disaggregated error by product revenue tier ===
1010 2         n_obs MAE WAPE mean_actual_demand
1011 3 tier
1012 4 Low (bottom third) 2000.0 133.297629 0.713058 186.9380
1013 5 Mid (middle third) 2000.0 82.728300 0.631494 131.0040
1014 6 High (top third) 2000.0 121.321784 0.523479 231.7605
1015 7
1016 8 === Dataset B: disaggregated error by VisitorType ===
1017 9         n_sessions actual_conv_rate FNR FPR
1018 10 VisitorType
1019 11 New_Visitor 429.0 0.256410 0.245455 0.050157
1020 12 Other 13.0 0.230769 0.000000 0.000000
1021 13 Returning_Visitor 2641.0 0.137827 0.354396 0.072025
1022 14
1023 15 === Dataset B: disaggregated error by Region ===
1024 16         n_sessions actual_conv_rate selection_rate FNR FPR
1025 17 Region
1026 18 1 1258.0 0.162957 0.163752 0.321951 0.063628
1027 19 2 269.0 0.171004 0.200743 0.304348 0.098655
1028 20 3 566.0 0.146643 0.160777 0.373494 0.080745
1029 21 4 303.0 0.158416 0.188119 0.250000 0.082353
1030 22 5 70.0 0.142857 0.200000 0.200000 0.100000
1031 23 6 204.0 0.132353 0.102941 0.518519 0.045198
1032 24 7 188.0 0.154255 0.132979 0.310345 0.031447
1033 25 8 110.0 0.081818 0.109091 0.444444 0.069307
1034 26 9 115.0 0.173913 0.182609 0.200000 0.052632

```

A.6 Reflection

The main process risk in Part 2 was that the original Task 1 code was unavailable, so the reconstruction could have quietly drifted from what was actually run in Task 1 without the author noticing. This was mitigated by validating every intermediate quantity (rows retained, panel shape, post-lag row count, held-out

metrics) against Task 1's own reported numbers before treating the reconstruction as a basis for new analysis, and by explicitly disclosing the one place a new issue was found (the `price_rel` look-ahead feature) rather than silently patching it. The bias analysis substituted a manually implemented equivalent for Fairlearn because the package could not be installed offline; this substitution is disclosed in Section 4.3 of the main text rather than presented as if Fairlearn itself had been used.

B Link to GitHub Repository

<https://github.com/Sahanaa-N/csds.git>